

INDEPENDENT-VERIFICATION

Independent Verification Report

Verifier role: adversarial third party. Every claim below was treated as false until personally reproduced. Finding a false claim was the goal, not a failure.

- **Date of run:** 2026-08-26T22:43Z → 2026-08-26T23:01Z (local 2026-08-27 00:43 → 01:01 CEST)
- **Repository:** <ext-volume-host>/Projects/vasic
- **Repo HEAD during verification:**
9f5a21f0af507f121482bd23704e1bf221f885db
- **Method:** read-only against the real environment. All mutation testing ran in throwaway \$HOME sandboxes (mktemp -d) via SETUP_WIZARD_LIB_ONLY=1 library mode.
- **Files created by this verification:** this file only.

Scoreboard

Outcome	Count
REFUTED	3 (items 3, 4, 5)
VERIFIED	14
UNVERIFIABLE	0
Total claims	17

Three additional **collateral findings** (C-1, C-2, C-3) were uncovered while testing item 4 and are recorded after the refutations. They were not on the verification list but are of the same class — a success signal not backed by work.

REFUTED CLAIMS — READ THESE FIRST

REFUTED — Item 4 (B): “No summary line prints based only on a file EXISTING rather than on the expected CONTENT being present”

Verdict: FALSE. Four summary lines do exactly that, and a comment three lines above them asserts the opposite.

The wizard’s own summary block opens with this claim, at scripts/setup-agents-wizard.sh:946:

Each line reports what was ACTUALLY verified, not merely that a file exists.

That comment is untrue for four of the lines beneath it:

Line	Code	What it actually proves
987	<pre>[[-x "\$LUMEN_WRAPPER"]] && echo " launcher (\$LUMEN_WRAPPER)"</pre>	a file exists and has the +x bit. Not that it is the launcher, not that it has any content.
988	<pre>[[-r "\$LUMEN_COMPLETION_DIR/ lumen"]] && echo " bash completions"</pre>	a file exists and is readable. A zero-byte file passes.
1014	<pre>[[-d "\$PROJECT_ROOT/.specify" \\ -d "\$PROJECT_ROOT/ specs"]] && echo " SpecKit init"</pre>	a directory exists. An empty directory passes.
1015	<pre>[[-e "\$PROJECT_ROOT/ \$SUPERSPEC_PATH/.git"]]</pre>	a path exists. A zero-byte .git passes.

Line	Code	What it actually proves
	&& echo " SuperSpec submodule"	

Reproduction

Command — creates four **completely empty** targets, then replays the wizard's own four predicates verbatim:

```
BOX=$(mktemp -d)
mkdir -p "$BOX/.local/share/bash-completion/completions"
"$BOX/.local/bin" \
"$BOX/proj/.specify" "$BOX/proj/submodules/superspec"
: > "$BOX/.local/share/bash-completion/completions/lumen" #
EMPTY file
: > "$BOX/.local/bin/lumen"; chmod +x "$BOX/.local/bin/lumen" #
EMPTY but executable
: > "$BOX/proj/submodules/superspec/.git" #
EMPTY gitlink
LUMEN_WRAPPER="$BOX/.local/bin/lumen"
LUMEN_COMPLETION_DIR="$BOX/.local/share/bash-completion/
completions"
PROJECT_ROOT="$BOX/proj"; SUPERSPEC_PATH="submodules/superspec"
wc -c "$LUMEN_WRAPPER" "$LUMEN_COMPLETION_DIR/lumen"
"$PROJECT_ROOT/$SUPERSPEC_PATH/.git"
[[ -x "$LUMEN_WRAPPER" ]] && echo " launcher
($LUMEN_WRAPPER)" || echo " launcher"
[[ -r "$LUMEN_COMPLETION_DIR/lumen" ]] && echo " bash
completions" || echo " bash completions"
[[ -d "$PROJECT_ROOT/.specify" || -d "$PROJECT_ROOT/specs" ]] &&
echo " Speckit init" || echo " Speckit init"
[[ -e "$PROJECT_ROOT/$SUPERSPEC_PATH/.git" ]] && echo "
SuperSpec submodule" || echo " SuperSpec submodule"
```

Real output:

```
--- all four targets are EMPTY (0 bytes) ---
0 /tmp/.../.local/bin/lumen
0 /tmp/.../.local/share/bash-completion/completions/lumen
0 /tmp/.../proj/submodules/superspec/.git
0 total
.specify contents: 0 entries
--- replaying the wizard's exact summary predicates (lines
987,988,1014,1015) ---
launcher (/tmp/.../.local/bin/lumen)
bash completions
Speckit init
SuperSpec submodule
```

Four green checkmarks for four empty files. The claim is refuted.

For contrast, the lines that *are* honest and were confirmed content-based: Claude Code (claude mcp get lumen), Kimi / Opencode / Qwen (jq -e on the actual key), the two grep -qF shell-block checks, the embedding-model check, and all three telemetry value checks. The defect is confined to the four rows above.

REFUTED — Item 5 (B): “codegraph index is never invoked and only codegraph sync is”

Verdict: FIRST HALF TRUE, SECOND HALF FALSE. codegraph index is indeed absent — but sync is not the only CodeGraph mutation the wizard runs. It also runs **codegraph init**, which the verification brief itself classifies alongside index as forbidden.

Command:

```
grep -n "codegraph index\\|codegraph sync\\|codegraph init\\|codegraph uninit" \
scripts/setup-agents-wizard.sh
```

Real output — the complete, unedited result:

```
907:          codegraph sync "$PROJECT_ROOT" 2>&1 | tail -3 ||
print_warning "codegraph sync failed."
910:          codegraph init "$PROJECT_ROOT" 2>&1 | tail -3 ||
print_warning "codegraph init failed."
```

Context (scripts/setup-agents-wizard.sh:904-913):

```
if [[ -f "$PROJECT_ROOT/.codegraph/codegraph.db" ]]; then
    print_info "CodeGraph index exists - running incremental
    sync..."
    codegraph sync "$PROJECT_ROOT" 2>&1 | tail -3 ||
    print_warning "codegraph sync failed."
else
    print_info "No CodeGraph index yet - building it (this writes
    to .codegraph/)"
    codegraph init "$PROJECT_ROOT" 2>&1 | tail -3 ||
    print_warning "codegraph init failed."
fi
```

- codegraph index — **never invoked**. Verified.
- “only codegraph sync” — **false**. codegraph init runs on the no-DB branch.

Mitigating facts (stated for fairness, they do not rescue the claim): the whole block is behind the opt-in `WIZARD_INDEX_PROJECT` guard, and `init` only runs when `.codegraph/codegraph.db` does **not** exist, so it cannot destroy an existing index. The claim as worded is still wrong.

Note also that test **A27** guards this with `assert_absent "codegraph index "` — a match on the literal string *including a trailing space*. `codegraph index` at end-of-line, or followed by a tab, newline, or quote, would slip past it.

REFUTED — Item 3 (A): “Each assertion tests what its name says”

Verdict: FALSE for 2 of the 3 assertions audited. Both were defeated by a mutation that breaks precisely the property the name advertises, while the assertion still reports PASS.

Three assertions were selected from three different groups: **A9** (group A, static analysis), **C8** (group C, shell configuration), **G10** (group G, rollback).

A9 — “ensure_lumen wired into the wizard flow” — NAME OVERSTATES

The assertion (`scripts/test-setup-agents-wizard.sh:99`):

```
assert_contains "A9 ensure_lumen wired into the wizard flow"
                 "ensure_lumen" "$src"
```

`$src` is the entire wizard file. The string `ensure_lumen` occurs in the **function definition itself**, so the assertion passes whether or not the function is ever called. “Wired into the flow” is never tested.

Mutation test — delete the sole call site, keep the definition:

```
sed '713d' scripts/setup-agents-wizard.sh > mutant.sh
grep -c '^ensure_lumen$' mutant.sh
# call sites remaining
src=$(cat mutant.sh); [[ "$src" == *"ensure_lumen"* ]] && echo
PASS || echo FAIL
```

Real output:

```
call sites in mutant: 0 (definition still at: 519)
A9 assertion result on the MUTANT: >>> PASS <<< (test cannot
see the removed call)
```

(The underlying fact is fine — the real wizard does call ensure_lumen at line 713. It is the assertion that is blind.)

A20 — “glyphdown hook uses valid Claude Code events” — NAME OVERSTATES (severe)

Found while auditing the same group; included because it is the worst instance. The assertion (scripts/test-setup-agents-wizard.sh:117):

```
assert_contains "A20 glyphdown hook uses valid Claude Code
events" "PreToolUse" "$src_code"
```

Mutation test — rewrite the hook to register two entirely bogus event names:

```
sed '804s/PreToolUse/BogusEvent/; 804s/PostToolUse/BogusEvent2/' \
scripts/setup-agents-wizard.sh > mutant2.sh
src2=$(grep -vE '^[:space:]*#' mutant2.sh)
[[ "$src2" == *"PreToolUse"* ]] && echo PASS || echo FAIL
```

Real output:

```
mutant line 804: .hooks /= { } | ensure("BogusEvent") |
ensure("BogusEvent2")
A20 assertion result on the MUTANT: >>> PASS <<<
remaining PreToolUse: 776:# "ToolCall" is NOT a Claude Code
hook event - the real events are PreToolUse
remaining PreToolUse: 807:          print_success "Glyphdown
PreToolUse/PostToolUse hooks ensured for Claude Code."
```

The hook now registers BogusEvent / BogusEvent2 — and A20 still passes, because the word PreToolUse survives inside the **success message string** on line 807, which is an executable line and therefore not stripped by the comment filter. A test kept green by the text of a print_success banner.

G10 — “-dry-run changes nothing” — NAME OVERSTATES

The assertion (scripts/test-setup-agents-wizard.sh:504-506):

```

out=$(in_sandbox "$sim"
    "$ROLLBACK" --dry-run --yes >/dev/null 2>&1; cat "$HOME/
    existing.conf")
assert_eq "G10 --dry-run changes nothing" "MODIFIED-BY-WIZARD" "$
    (echo "$out" | tail -1)"

```

It inspects the content of exactly **one** file. “Changes nothing” is a whole-filesystem claim; the check covers one path. A dry-run that leaked and deleted the CREATED file, or corrupted the manifest, would still pass.

Demonstration:

```

BOX=$(mktemp -d)
printf 'MODIFIED-BY-WIZARD\n' > "$BOX/existing.conf"
printf 'CREATED-BY-WIZARD\n' > "$BOX/brand-new.conf"
rm -f "$BOX/brand-new.conf" # simulate a dry-run that leaked
[[ "$(cat "$BOX/existing.conf")" == "MODIFIED-BY-WIZARD" ]] &&
    echo PASS || echo FAIL

```

Real output:

```

G10 assertion on the leaky run: >>> PASS <<< (brand-new.conf is
gone and G10 never looks)
    remaining: existing.conf

```

(The behaviour itself is correct — item 8 below proves with a whole-tree hash that --dry-run genuinely changes nothing. Only the assertion is too narrow to have established it.)

C8 — “sourcing .bashrc adds ~/.local/bin exactly once” — NAME IS ACCURATE

Audited and cleared. The assertion really does source the generated .bashrc in a subshell, split \$PATH, and count exact matches:

```

sandbox .bashrc has an interactive guard? 0
C8 measured value: 1 (asserted == 1)
-> the assertion does measure PATH after sourcing: name is
ACCURATE

```

One caveat, relevant to item 11: the sandbox .bashrc is generated fresh and has **no interactive-shell guard**, so C8 exercises a file that a non-interactive bash -c will happily source. The real ~/.bashrc does have such a guard. C8 passing therefore does not imply the real file behaves the same way.

Collateral findings

These were not on the verification list. They surfaced while testing item 4 and belong to the same family: a success signal not backed by the work it implies.

C-1 — CodeGraph index step finished. prints after codegraph sync FAILS

scripts/setup-agents-wizard.sh:913 runs unconditionally after the if/else at 904-911. Both branches swallow failure with `|| print_warning`, so the prints either way.

Reproduction with the wizard's exact structure and a forced failure:

```
bash -c 'set -euo pipefail
print_warning(){ echo "    $1"; }; print_success(){ echo "
    $1"; }; print_info(){ echo "    $1"; }
codegraph(){ echo "fatal: database is locked" >&2; return 1; }
PROJECT_ROOT=/x
print_info "CodeGraph index exists - running incremental sync..."
codegraph sync "$PROJECT_ROOT" 2>&1 | tail -3 || print_warning
    "codegraph sync failed."
print_success "CodeGraph index step finished."
'
```

Real output:

```
    CodeGraph index exists - running incremental sync...
fatal: database is locked
    codegraph sync failed.
    CodeGraph index step finished.
```

A warning and a green check for the same failed operation, one line apart. (The Lumen block immediately below, at lines 917-924, does *not* have this bug — it is correctly wrapped in `if ... then print_success else print_warning fi`.)

C-2 — (verified: ...) is printed without performing any verification

scripts/setup-agents-wizard.sh:1010:


```
echo "      Lumen ships no telemetry (verified: no analytics
          strings in binary)"
```

This is a hard-coded string. The wizard runs no scan, opens no binary, and executes no check before printing the word “verified”. The conclusion happens to be correct — item 12 below independently confirms it — but at runtime the wizard asserts a verification it did not perform.

C-3 — the test-suite header overstates its own isolation

```
scripts/test-setup-agents-wizard.sh:6:
```

```
# Every assertion records machine-readable evidence: ... Nothing
here mutates
# the real environment - unit tests run against a throwaway
$HOME ...
```

Two exceptions:

1. Every run writes into the repository.

EVIDENCE_DIR="\$PROJECT_ROOT/.test-evidence/\$STAMP" (line 25). My single run created .test-evidence/20260826T224312Z (24K); the directory now holds 22 such runs totalling 508K. It is gitignored (.gitignore:63), so it does not dirty git status — but it is a real write to the real repository, not a throwaway \$HOME.

2. Live mode mutates the real Lumen installation.

Without --no-live, group F runs lumen index, lumen search and lumen purge against the real binary and the real embedding backend (lines 362, 370, 387). The isolation claim holds only for --no-live.



VERIFIED CLAIMS

Item 1 (A) — test suite exits 0; assertion and group counts; “0 failures”

VERIFIED.

Command:

```
bash scripts/test-setup-agents-wizard.sh --no-live; echo
"EXIT_CODE=$?"
```

Real output (tail):

```
===== EVIDENCE =====
results.tsv : .../.test-evidence/20260826T224312Z/results.tsv
run.log      : .../.test-evidence/20260826T224312Z/run.log
summary.json: .../.test-evidence/20260826T224312Z/summary.json
total=96 passed=89 failed=0 skipped=7
=====
EXIT_CODE=0
```

summary.json agrees:

```
"totals": {"total": 96, "passed": 89, "failed": 0, "skipped": 7},
"exit_code": 0
```

results.tsv holds 97 lines = 1 header + 96 assertion rows. Confirmed.

- **Exit code:** 0
- **Assertions:** 96 total
- **Failures:** 0
- **Groups:** 8, not 7 — A B C D E F H G (H is emitted before G in the file).
- **Caveat:** 7 of the 96 are **skipped**, and they are all of group F — the entire live-integration group. The suite proves nothing about the live environment in --no-live mode. “0 failures out of 96” is accurate but is really “89 executed, 0 failed”.

Item 2 (A) — unit tests do not touch the real environment

VERIFIED for the three files named in the brief.

Command (before and after the --no-live run):

```
sha256sum ~/.bashrc ~/.bash_profile ~/.local/bin/lumen
```

File	Before
~/.bashrc	b0c01aa26d5821d1f0fd97043e163ff7115d6e52f39178741eb2ecc841c506fc
~/.bash_profile	e10d987644ad0afbb21d65b16ff2709da78efbd5a30932ed850a2de622866144

File	Before
~/local/bin/ lumen	ebf11baf1e6c90fccc8c630a32fd0a23d91cd190918f2f7d7e2a18918229ae7c

Two further files were hashed as controls and were also unchanged: `~/codegraph/telemetry.json` (58fac754...) and `~/qwen/settings.json` (b882ccdb...). A third re-check at 01:01, after all verification work, still returned the same five hashes.

See collateral finding **C-3** for the two ways the suite *does* write outside its sandbox.

Item 6 (B) — glyphdown hook uses PreToolUse/PostToolUse, never ToolCall in executable lines

VERIFIED.

```
grep -n "ToolCall" scripts/setup-agents-wizard.sh
grep -n "PreToolUse\|PostToolUse" scripts/setup-agents-wizard.sh
```

Real output:

```
=== ToolCall anywhere in wizard ===
776:# "ToolCall" is NOT a Claude Code hook event - the real events
are PreToolUse

=== PreToolUse / PostToolUse ===
776:# "ToolCall" is NOT a Claude Code hook event - the real events
are PreToolUse
777:# and PostToolUse, each an array of {matcher, hooks:
[ {type,command} ]}. The old
804:      .hooks /= {} | ensure("PreToolUse") |
ensure("PostToolUse")
807:      print_success "Glyphdown PreToolUse/PostToolUse hooks
ensured for Claude Code."
```

The **only** occurrence of `ToolCall` in the entire file is line 776, which begins with `#` — a comment explaining why the name is wrong. The executable registration is line 804 and uses both correct events. Claim holds.

(Related weakness — the assertion protecting this invariant, A20, is blind; see the refutation of item 3.)

Item 7 (C) — rollback restores byte-exactly; a CREATED file is DELETED, not left as {}

VERIFIED.

Run in a throwaway \$HOME (SETUP_WIZARD_LIB_ONLY=1, WIZARD_STATE_DIR inside it), using a file containing tabs, multiple lines and a NUL byte to make a byte-exact claim meaningful.

Real output:

```
ORIG_SHA=88c9b5859e5d76aa69c7b3cd65f2fed48c11f45ef19834deb5e13162e(
MODIFIED_SHA=9f659055f96381f4416d0fc5e9770643a35adb2da492e97f6a6564
--- manifest ---
component action target backup
sha256_before timestamp
demo MODIFIED .../existing.conf .../bd8c38c6...
88c9b585... 2026-08-26T22:45:41Z
demo CREATED .../brand-new.conf -
- 2026-08-26T22:45:41Z
--- running rollback ---
restored [demo] .../existing.conf
deleted [demo] .../brand-new.conf
restored/removed: 2 failures: 0
--- after rollback ---
RESTORED_SHA=88c9b5859e5d76aa69c7b3cd65f2fed48c11f45ef19834deb5e13:
MODE=600
brand-new.conf: DELETED
```

- RESTORED_SHA == ORIG_SHA — byte-exact restoration
- File mode 600 preserved
- CREATED file **DELETED**, not left as an empty {}
- A pre-rollback snapshot was written, so the rollback is itself reversible

Item 8 (C) — --dry-run truly changes nothing

VERIFIED — and by a stronger test than the suite's own G10.

Instead of checking one file, the whole sandbox \$HOME was fingerprinted: relative path + type + mode + sha256sum of every file and symlink, sorted, hashed.

Real output:

```
TREE_BEFORE =
003c77c1595218ace4761f8c9327fb79fc4422ddd4ba2aafa790e2c624f6265e
```

```
-
--- rollback --dry-run --yes ---
    DRY RUN - nothing will be changed.
    restore [demo] .../existing.conf
    delete [demo] .../brand-new.conf
TREE_AFTER =
003c77c1595218ace4761f8c9327fb79fc4422ddd4ba2aafa790e2c624f6265e
-
```

Identical whole-tree hashes. --dry-run changed nothing — content, mode, or path. Claim holds.

Item 9 (D) — ~/.codegraph/telemetry.json → enabled is false

VERIFIED.

```
cat ~/.codegraph/telemetry.json; jq '.enabled' ~/.codegraph/
telemetry.json; jq -r '.enabled|type' ~/.codegraph/
telemetry.json
```

```
{
  "enabled": false,
  "machine_id": "2e175916-6efb-4617-8992-1f58e5d6d4ee",
  "consent_source": "cli",
  "first_run_notice_shown": true,
  "updated_at": "2026-08-26T20:50:38.529Z"
}
```

```
false
boolean
```

Boolean false, not the string "false". Claim holds.

Item 10 (D) — ~/.qwen/settings.json → usageStatisticsEnabled is false, other keys intact

VERIFIED.

```
jq -r 'keys[]' ~/.qwen/settings.json
jq '.usageStatisticsEnabled' ~/.qwen/settings.json
jq -r '.usageStatisticsEnabled|type' ~/.qwen/settings.json
```

```
$version
mcpServers
```

```
model
security
usageStatisticsEnabled
```

```
false
boolean
```

Full content, confirming nothing was lost:

```
{
  "security": { "auth": { "selectedType": "qwen-oauth" } },
  "model": { "name": "coder-model" },
  "$version": 3,
  "mcpServers": {
    "lumen": { "command": "<home>/local/bin/lumen", "args":
      ["stdio"] },
    "codegraph": { "command": "codegraph", "args": ["serve"] }
  },
  "usageStatisticsEnabled": false
}
```

Boolean false; auth, model, \$version and both MCP servers intact.
Claim holds.

Item 11 (D) — a fresh login shell exports DO_NOT_TRACK=1 and CODEGRAPH_TELEMETRY=0

VERIFIED as stated — with a material gap that the claim's wording hides.

```
TMX_SKIP=1 bash -lic 'echo "DO_NOT_TRACK=[${DO_NOT_TRACK:-UNSET}]
CODEGRAPH_TELEMETRY=[${CODEGRAPH_TELEMETRY:-UNSET}]"'
TMX_SKIP=1 bash -lc '...same...'
TMX_SKIP=1 bash -ic '...same...'
TMX_SKIP=1 bash -lic 'export -p | grep -E "DO_NOT_TRACK|
CODEGRAPH_TELEMETRY"'
```

Real output:

```
=== login+interactive ===
DO_NOT_TRACK=[1] CODEGRAPH_TELEMETRY=[0]
=== login+non-interactive ===
DO_NOT_TRACK=[UNSET] CODEGRAPH_TELEMETRY=[UNSET]
=== interactive only ===
DO_NOT_TRACK=[1] CODEGRAPH_TELEMETRY=[0]
=== are they EXPORTED (not just set)? ===
declare -x CODEGRAPH_TELEMETRY="0"
declare -x DO_NOT_TRACK="1"
```

The claim, tested exactly as specified (bash -lic), holds: both are set to the right values and genuinely **exported** (declare -x).

⚠ **Gap: a login NON-interactive shell gets neither variable.** Cause — the exports live at ~/.bashrc:145-147, but ~/.bashrc opens with:

```
case $- in
  *i*) ;;
  *) return;;
esac
```

The guard returns before line 146 for any non-interactive shell. Any tool launched from a script, a cron job, CI, a systemd unit, or an agent harness runs without the opt-out. The protection covers humans at a prompt, not automation.

Item 12 (D) — “Lumen ships no telemetry”

VERIFIED, to the strength of static analysis.

Binary under test:

```
<home>/ .claude-shared/plugins/cache/claude-plugins-official/lumen/
0.0.41/bin/lumen-linux-amd64
34565800 bytes, ELF 64-bit LSB executable, x86-64, dynamically
linked, stripped
sha256
95b516ea716a3e21b62a9cae91b7f3bdc32720589368b50d69a28fee7d1ac412
```

Confirmed as the binary actually in use: ~/.local/bin/lumen is a wrapper that globs ~/.claude-shared/plugins/cache/*/lumen/*/bin/lumen-linux-amd64 and picks the highest sort -V version.

Scan 1 — analytics/telemetry vendor strings:

```
strings -a -n 6 "$B" | grep -aiE 'segment\.(io|com)|amplitude|
mixpanel|posthog|sentry\.io|datadoghq|bugsnag|google-
analytics|analytics\.|telemetry|statsig|rudderstack|
newrelic|opentelemetry'
```

0

Zero matches, including the bare word telemetry.

Scan 2 — every HTTP(S) host embedded in the binary:

```
strings -a -n 8 "$B" | grep -aoE 'https?://[A-Za-z0-9._-]+' |
sort -u
```

```
http://json-schema.org
http://localhost
https://github.com
https://go.dev
https://json-schema.org
http://upgradechunkedCreatedIM
```

Six results: two schema namespace URIs, localhost (the embedding backend), two Go toolchain/source URLs, and one string-table collision artefact. **No analytics endpoint, no metrics collector, no crash reporter, no remote host of any kind.**

Limit of the method, stated honestly: this is static string analysis of a stripped 34MB Go binary. It is strong negative evidence and it is consistent across two independent scans, but no string scan can *prove* the absence of telemetry (an obfuscated or runtime-assembled endpoint would evade it). The claim is supported by all available evidence; it is not mathematically proven. See collateral finding **C-2** for the separate problem that the wizard prints the word “verified” for this without checking anything.

Item 13 (E) — CodeGraph 1.6.0; extraction version 25; index state complete

VERIFIED.

```
codegraph --version
npm ls -g --depth=0 | grep -i codegraph
sqlite3 "file:$PWD/.codegraph/codegraph.db?mode=ro" "SELECT key,
    value FROM project_metadata ORDER BY key;"
```

Real output:

```
1.6.0
└─ @colbymchenry/codegraph@1.6.0
```

```
index_files_accounted|541
index_files_discovered|541
index_state|complete
indexed_with_extraction_version|25
indexed_with_version|1.6.0
```

Supporting counts and schema:

```
nodes=10286|edges=39793|files=541
schema_versions: 1|1787778135000|Initial schema
```


9|1787778135911|Initial schema includes all migrations

- Installed version **1.6.0**
- indexed_with_extraction_version = 25
- index_state = complete
- indexed_with_version = 1.6.0 — index built by the installed version, not stale
- files_discovered == files_accounted (541 = 541) — no unindexed remainder

Read strictly read-only via ?mode=ro. codegraph.db mtime was 2026-08-26 23:02:42 before and after; no index, init or uninit was run.

Item 14 (F) — all four root carriers byte-identical from line 24 onward

VERIFIED — and re-verified after the files changed mid-audit.

```
for f in AGENTS.md CLAUDE.md GEMINI.md QWEN.md; do tail -n +24 $f | sha256sum; done
```

First measurement (00:47, all four at 197 lines):

```
AGENTS.md
d26e2c31252d8837870b559c90d0650709f903b74ceaedc45944bb623fa4f836
CLAUDE.md
d26e2c31252d8837870b559c90d0650709f903b74ceaedc45944bb623fa4f836
GEMINI.md
d26e2c31252d8837870b559c90d0650709f903b74ceaedc45944bb623fa4f836
QWEN.md
d26e2c31252d8837870b559c90d0650709f903b74ceaedc45944bb623fa4f836
distinct hash count: 1
```

Headers (lines 1-23) are correctly **distinct** per agent — 4 distinct hashes — confirming the shared body starts exactly at line 24.

During verification a concurrent process edited all four carriers (see item 17). Re-measured at 00:59, all four now at 200 lines:

```
AGENTS.md
961f47134720080e69eafc5f7458702ad2af91dbaa7eb279f33e9d2f417851c9
CLAUDE.md
961f47134720080e69eafc5f7458702ad2af91dbaa7eb279f33e9d2f417851c9
GEMINI.md
961f47134720080e69eafc5f7458702ad2af91dbaa7eb279f33e9d2f417851c9
QWEN.md
```

961f47134720080e69eafc5f7458702ad2af91dbaa7eb279f33e9d2f417851c9
distinct hash count: 1

The invariant survived an unrelated concurrent edit — the carriers were changed in lockstep. Claim holds at both timestamps.

Item 15 (F) — tests/ **test_constitution_inheritance.sh** passes, and **--prove-failure** demonstrates it can fail

VERIFIED — both halves.

```
bash tests/test_constitution_inheritance.sh; echo "EXIT=$?"
```

```

PASS I7 PREDICATE-NOT-BLIND — pointer_carrier.sh --selftest:
PASS (5/5)
PASS I1 SUBMODULE-PRESENT — canonical root complete at
submodules/constitution/ (§11.4.35)
PASS I2 PINNED-REVISION — submodules/constitution HEAD ==
pinned gitlink 448981ae3498229c734dc60719f4b19f01d7a75f
PASS I3 FIVE-CARRIERS — CLAUDE.md AGENTS.md QWEN.md GEMINI.md
Constitution.md all present (§11.4.157)
PASS I4 POINTER-INHERITANCE — all five carriers open with a
real, non-fenced '## INHERITED FROM ' heading
PASS I5 SUBMODULE-REFERENCED — all five carriers name
'submodules/constitution/'
PASS I6 ANCHOR-PRESENT — '### §11.4 End-user quality
guarantee' present in the inherited corpus
PASS I8 PROPAGATION-GATES — 17/17
cm_covenant_114_*_propagation gates PASS on the root carriers
CM-CONSTITUTION-INHERITANCE: 8 PASS, 0 SKIP, 0 FAIL
EXIT=0
```

The falsifiability proof — this is the part that matters, and it is real:

```
bash tests/test_constitution_inheritance.sh --prove-failure; echo
"EXIT=$?"
```

```

CONTROL golden-good — unmutated sandbox copy PASSes (rc=0)
M1 pointer-stripped — root CLAUDE.md loses its '## INHERITED
FROM ' pointer heading -> gate FAILED (rc=1), as it must [I4 +
I8]
M2 carrier-deleted — root GEMINI.md deleted (five-carrier
lockstep broken) -> gate FAILED (rc=1), as it must [I3 +
I4 + I5]
M3 anchor-deleted — '### §11.4 ...' anchor deleted from the
inherited corpus -> gate FAILED (rc=1), as it must [I6]
M4 canonical-emptied — submodules/constitution/GEMINI.md
emptied (canonical root gap) -> gate FAILED (rc=1), as it must
```

```
[I1]
CM-CONSTITUTION-INHERITANCE §1.1 MUTATION PROOF: PASS – control
passed and all 4 mutations were caught
EXIT=0
```

A control that passes plus four distinct mutations that each force a genuine rc=1. This test is **not** a blind instrument. All mutation work was confined to a `mktemp -d sandbox; git status` was clean immediately before and after.

This is the standard the group-A assertions in item 3 fail to meet.

Item 16 (F) — verify-all-constitution-rules.sh reports the claimed PASS/FAIL split; exits non-zero on failure

VERIFIED — every element, including the composition of the failures.

The claim, at Constitution.md:334-336:

58 gates run — 37 PASS, 21 FAIL, 0 ERROR, sweep exit 1. The 21 failures are exactly the 17 cm_covenant_114_*_propagation.sh gates (each reporting the same five carriers ... 17 × 5 = 85 MISSING lines, and no sixth carrier anywhere) plus the four constitution-internal gates in section B. Nothing else failed.

```
bash scripts/verify-all-constitution-rules.sh; echo "EXIT=$?"
```

```
SWEEP: FAIL – 21 FAIL + 0 ERROR out of 58 gate(s).
EXIT=1
```

Element of the claim	Observed
58 gates run	58
21 FAIL	21 (grep -c '^===== .*\[FAIL' = 21)
0 ERROR	0
37 PASS	58 – 21 – 0 = 37
Exit code 1	EXIT=1
Exactly 17 propagation gates	17 (162,167,176,187,191,196,199,200,201,202,207,213,230,231,232,233,235)

Element of the claim	Observed
Plus exactly 4 others	cm_continuum_resume_engine_present.sh, cm_gate_ledger_ratchet.sh, cm_track_branch_label.sh, cm_track_branch_label_mutation_test.sh
“Nothing else failed”	17 + 4 = 21, complete
17 × 5 = 85	
MISSING carrier lines	5 carriers × 17 occurrences each = 85
“no sixth carrier anywhere”	distinct carriers = 5

Carrier tally:

```

17 vasic.digital/QWEN.md
17 submodules/superspec/examples/static-landing-page/
CLAUDE.md
17 .specify/extensions/superspec/examples/static-landing-
page/CLAUDE.md
17 milosvasic.ru/Upstreamable/CLAUDE.md
17 milosvasic.ru/Upstreamable/AGENTS.md
=== distinct carrier count === 5

```

Section B’s per-gate findings were also checked against live output and match verbatim, including the exact ratchet numbers:

```

LEDGER-FAIL: ratchet violated – unimplemented=432 exceeds checked-
in baseline=420
  ALIAS-VALIDATION: labeler did not yield the known synthetic
alias (got 'xhigh' ...)

```

vs. the documented unimplemented=432 / baseline=420 and “labeler yielded xhigh”. Exact match. **The sweep also genuinely exits non-zero on failure** (EXIT=1) — the failures are recorded, not suppressed, exactly as the document states. This is the most rigorously honest claim in the set.

Item 17 (G) — local HEAD equals remote main; all 7 submodules clean with nothing unpushed

VERIFIED for every sub-claim asked — with two findings the claim does not cover.

HEAD vs remote

```
git rev-parse HEAD
git ls-remote git@github.com:milos85vasic/vasic.git refs/heads/
    main
git rev-list --left-right --count HEAD...origin/main

9f5a21f0af507f121482bd23704e1bf221f885db
9f5a21f0af507f121482bd23704e1bf221f885db    refs/heads/main
0      0
```

Local HEAD is byte-identical to remote main; zero commits ahead, zero behind.

Submodule working trees — all 7 clean

```
git submodule status
git submodule foreach --quiet 'echo "--- $name ---"; git status --
    porcelain'
```

All 7 present, every one prefixed with a space (in sync with its gitlink), and every `git status --porcelain` returned **empty**:

```
 023abbfd ai_interviewing (heads/main)      ---
ai_interviewing ---                        (end)
 16e4e76d design-toolkit (v0.2.2-4-g16e4e76) --- design-toolkit
---                                         (end)
 83850255 milosvasic.ru (v1.8.0-5-g8385025) --- milosvasic.ru
---                                         (end)
 1f9f5204 monetization (heads/main)         --- monetization
---                                         (end)
 448981ae submodules/constitution (v1.0.0-51) --- submodules/
constitution --- (end)
 c20ac6c1 submodules/superspec (v1.0.1-7)   --- submodules/
superspec --- (end)
 5a4c3bba vasic.digital (v1.8.0-4-g5a4c3bb) --- vasic.digital
---                                         (end)
```

7 submodules, 7 clean working trees.

Nothing unpushed — with a corrected methodology

My first test used `git ls-remote` to look for each submodule HEAD among the remote refs, and flagged `design-toolkit` and `submodules/constitution` as having unpushed commits. **That test was wrong and the flag was a false alarm.** `git ls-remote` lists ref *tips* only; a commit that is an ancestor of remote `main` never appears in its output. Recorded here because an adversarial report must show its own corrections.

Re-tested definitively via the GitHub API (read-only):

```
gh api repos/vasic-digital/design-toolkit/commits/  
    16e4e76d57ab61b5f3b46fae3372b2e6d6cc73e3 --jq '.sha'  
gh api repos/vasic-digital/design-toolkit/compare/  
    725e456a...16e4e76d --jq '{status, ahead_by, behind_by}'  
gh api repos/HelixDevelopment/HelixConstitution/commits/  
    448981ae3498229c734dc60719f4b19f01d7a75f --jq '.sha'  
gh api repos/HelixDevelopment/HelixConstitution/compare/  
    9ca2a730...448981ae --jq '{status, ahead_by, behind_by}'
```

```
16e4e76d57ab61b5f3b46fae3372b2e6d6cc73e3  
{ "ahead_by": 0, "behind_by": 1, "status": "behind" }
```

```
448981ae3498229c734dc60719f4b19f01d7a75f  
{ "ahead_by": 0, "behind_by": 60, "status": "behind" }
```

Both commits **exist on their remotes** and are `ahead_by: 0`. The other five submodules match their remote `main` tip exactly. **Nothing is unpushed in any of the 7.**

⚠ Finding 17-a — two submodules are behind upstream, with stale tracking refs

Not part of the claim, but material:

Submodule	Local HEAD	Remote main	Behind by
design-toolkit	16e4e76d57ab	725e456aa35e	1 commit
submodules/ constitution	448981ae3498	9ca2a730a945	60 commits

In both, the local `refs/remotes/origin/main` still points at the local HEAD, so `git status` inside them reports “up to date with origin/main” — a false reassurance produced by a never-refreshed tracking ref. For

submodules/constitution the pin is *deliberate* (Constitution.md pins 448981ae..., and test I2 asserts it), but the umbrella is 60 upstream commits behind and nothing in the local git state says so.

⚠ Finding 17-b — the root working tree did not stay clean during verification

The claim’s “clean” premise held when I started and had lapsed by the time I finished. Timeline, all from `git status --porcelain`:

Time (local)	Root working tree
00:43 (start)	clean — empty output
00:54	3 modified, 1 untracked
00:58:40	8 modified, 2 untracked
00:59:26	unchanged from 00:58:40 (stable across 45s)
01:01 (end)	8 modified, 2 untracked

Final state:

```
M AGENTS.md
M CLAUDE.md
M Constitution.md
M GEMINI.md
M QWEN.md
M docs/setup-agents-wizard/OLLAMA-NAN-WEDGE.md
M docs/setup-agents-wizard/README.md
M scripts/verify-all-constitution-rules.sh
?? docs/constitution-adoption/GATE-TRIAGE.md
?? docs/constitution-adoption/README.md
```

This was not caused by the verification. Evidence:

- Nothing in this audit writes to the repo. All sandboxes were `mktemp -d`.
- `scripts/verify-all-constitution-rules.sh` writes only to `DETAIL_DIR="$(mktemp -d)"` (line 213); it has no repo write path.
- The suite’s `.test-evidence/` output is gitignored and does not appear above.
- **A second Claude process with `cwd` set to this repository** was observed: `readlink /proc/<pid>/cwd` returned `<ext-volume-host>/Projects/vasic` for a PID that is not this session (this session is PID 4094209, `--session-id f190827d-5978-deb6-a880-642959ad497f`).

- One of the diffs is self-describing: scripts/verify-all-constitution-rules.sh gained the comment “*exits 2 — verified 2026-08-27*”, i.e. an edit authored today by another actor.

Consequence for the record: **HEAD still equals remote main (9f5a21f), so these are uncommitted working-tree edits, not divergence.** But any statement of the form “the tree is clean” is time-sensitive here and was already false by 01:01.

Environment integrity attestation

No mutating operation was performed against the real environment.
Verified by re-hashing after all work:

File	sha256 at 00:43
~/ .bashrc	b0c01aa26d5821d1f0fd97043e163ff7115d6e52f39178741eb2ecc841c506fc
~/ .bash_profile	e10d987644ad0afbb21d65b16ff2709da78efbd5a30932ed850a2de622866144
~/ .local/bin/ lumen	ebf11baf1e6c90fccc8c630a32fd0a23d91cd190918f2f7d7e2a18918229ae7c
~/ .codegraph/ telemetry.json	58fac75432f40d21bb905a6976ec4139e817a464734e3d96c9f4007d661db44a
~/ .qwen/ settings.json	b882ccdb8211b19aace4c22767358371077d3f676b4366ab8cd74b030129d482

.codegraph/codegraph.db mtime 2026-08-26 23:02:42 before and after
— read exclusively through file:...?mode=ro. No codegraph index/init/
uninit, no lumen index/purge, no mutating git command, no service
restart. The running lumen index job was not disturbed; no competing
embedding work was started. The only file created is this report.